# Convergence Analysis of Gradient Descent Optimization

Daniel Ari Friedman
Active Inference Institute
daniel@activeinference.institute
ORCID: 0000-0001-6232-9096
DOI: 10.5281/zenodo.20417136

2026-06-26

## BEGINNING OF TRANSMISSION

**State:** unpublished / pending pairing
**Pairing:** pending — unresolved: - GitHub release URL: `pending` - PDF SHA-256: `pending`

### Release metadata

| Field | Value |
|---|---|
| Title | Convergence Analysis of Gradient Descent Optimization |
| Version | 3.0.0 |
| Concept DOI | 10.5281/zenodo.20417136 |
| GitHub | docxology/template_code_project |
| Zenodo | https://zenodo.org/records/20417136 |
| SHA-256 | pending |
| SHA-512 | pending |

### How to verify

- Scan **Integrity** QR and compare the embedded SHA-256 prefix to the table above.
- Scan **Zenodo** / **GitHub** QR codes and confirm they resolve to this release pairing.
- Full hashes and structured fields: `../data/transmission_manifest.json`.




Figure 1: Integrity QR strip

Structured manifest: `../data/transmission_manifest.json`
**Stego:** on | overlays text | barcodes on | XMP on | manifest on → `./secure_run.sh`

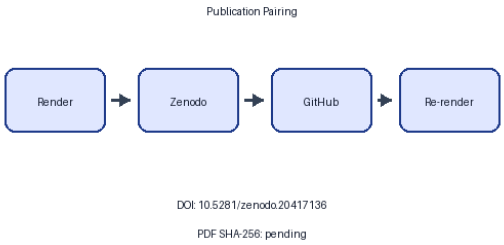



Figure 2: Publication pairing flow

# Contents

**1 Abstract** **4**

**2 Introduction** **5**
2.1 Template Architecture Context . . . 5
2.2 Infrastructure Integration . . . 5
2.3 Algorithm Overview . . . 5
2.4 Exemplar Implementation Goals . . . 5
2.5 Reader's guide to the manuscript . . . 5
2.6 Why a quadratic model . . . 6

**3 Methodology** **7**
3.1 Algorithm Implementation . . . 7
3.1.1 Gradient Descent Algorithm . . . 7
3.2 Infrastructure Integration . . . 7
3.2.1 Numerical Stability Analysis . . . 7
3.2.2 Performance Benchmarking . . . 7
3.3 Convergence Analysis . . . 7
3.4 Experimental Setup . . . 7
3.4.1 Step Size Analysis . . . 7
3.4.2 Zero-Mock Testing Methodology . . . 8
3.4.3 Stopping rule and reporting . . . 8
3.4.4 Figure generation contract . . . 8
3.5 Analysis Pipeline & LaTeX Integration . . . 8

**4 Results** **9**
4.1 Convergence Analysis . . . 9
4.1.1 Convergence Trajectories . . . 9
4.1.2 Step Size Sensitivity Analysis . . . 10
4.2 Quantitative Results . . . 10
4.3 Convergence Rate Analysis . . . 10
4.3.1 Theoretical vs Empirical Convergence . . . 10
4.3.2 Error Bounds . . . 12
4.3.3 Performance Metrics . . . 12
4.4 Performance Analysis . . . 12
4.4.1 Convergence Speed . . . 12
4.4.2 Solution Accuracy . . . 12
4.5 Algorithm Characteristics . . . 12
4.5.1 Strengths . . . 12
4.5.2 Limitations . . . 13
4.6 Computational Performance . . . 13
4.6.1 Algorithm Complexity Visualization . . . 13
4.6.2 Performance Benchmarking . . . 13
4.6.3 Numerical Stability Analysis . . . 13
4.6.4 Performance Metrics Summary . . . 13
4.7 Validation . . . 13
4.8 Discussion . . . 16

**5 Conclusion** **17**
5.1 Exemplar Project Achievements . . . 17
5.2 Technical Contributions . . . 17
5.2.1 Test Coverage Strategy . . . 17
5.2.2 Infrastructure-Backed Capabilities . . . 17
5.3 Research Pipeline Validation . . . 17
5.4 Key Insights . . . 17
5.5 Future Extensions . . . 17
5.6 Final Assessment . . . 18

**6 Experimental Setup** **19**
6.1 Problem Definition . . . 19
6.2 Parameter Space . . . 19
6.3 Numerical Stability Grid . . . 19
6.4 Dimensional Scaling . . . 19
6.5 Computational Environment . . . 19
6.6 Pipeline ordering . . . 19
6.7 Relation to figures . . . 19

**7 Reproducibility Certification** **21**
7.1 Configuration Provenance . . . 21
7.2 Generated Artifact Registry . . . 21
7.3 Numerical Validation Summary . . . 21
7.3.1 Convergence Verification . . . 21
7.3.2 Numerical Stability . . . 21
7.3.3 Benchmark Demonstration . . . 21
7.4 Madlib Injection Verification . . . 22

**8 Scope, Related Work, and Positioning** **23**
8.1 Classical gradient methods . . . 23
8.2 Adaptive and stochastic extensions . . . 23
8.3 What this project proves about the template . . . 23
8.4 Explicit limitations . . . 23

**9 References** **24**

# 1 Abstract

This paper presents a convergence study of **fixed-step gradient descent** on a convex quadratic, framed as the computational exemplar of the Research Project Template. The implementation lives in `projects/templates/template_code_project/src/template_code_project/core/optimizer.py`; experiments and figures are orchestrated by `projects/templates/template_code_project/scripts/optimization_analysis.py` and hydrated into the manuscript through `scripts/z_generate_manuscript_variables.py`, so tables and prose track `output/data/optimization_results.csv` after every pipeline run.

We evaluate 6 step sizes from $\alpha = 0.01$ to $\alpha = 2.5$, spanning conservative, near-optimal, aggressive, and divergent regimes for a unit Hessian model. The build chain exercises template infrastructure end-to-end: scientific helpers (`infrastructure.scientific.stability`, `infrastructure.scientific.benchmarking`), validation, rendering (`infrastructure/rendering/pdf_renderer.py`), and reporting. Accessibility-oriented plotting defaults (colourblind-safe palette, 300 dpi exports) are centralized in `src/template_code_project/figures/` and `src/template_code_project/analysis/`.

Contributions are **methodological** and **architectural**. On the methods side, we relate empirical iteration counts and error decay to the scalar contraction factor $\rho(\alpha) = |1-\alpha|$ and document cases where runs hit $N_{\max} = 1000$ before meeting the gradient tolerance. On the architecture side, we demonstrate a zero-mock test suite on project `src/` (see test_optimizer.py), automated six-figure analysis, and reproducibility metadata (configuration hash, artifact counts) injected into sec. 7.

**Results (this configuration):** 4 of 6 grid points report `converged=True` in the CSV; non-convergent rows flag either slow progress at small $\alpha$ under the iteration cap or instability when $|1-\alpha| \geq 1$. The analytical minimizer remains $x^* = 1.0$ with $f(x^*) = -0.5$ for the configured $(A, b)$.

**Keywords:** optimization algorithms, gradient descent, convergence analysis, numerical methods, mathematical programming, reproducible research, infrastructure automation

# 2 Introduction

This `template_code_project` serves as the foundational exemplar for the Research Project Template ecosystem, demonstrating a fully-tested numerical optimization implementation bracketed by rigorous infrastructure, hermetic testing, and extensive documentation architectures. The prose, the labelled figures, and the labelled equations have all been generated through an auditable custody chain starting from algorithm implementation through strict CI/CD validation to multi-format `.pdf` compilation.

## 2.1 Template Architecture Context

Scientific engineering requires mathematical accuracy combined with software reliability. This project unifies theoretical optimization with the repository's three foundational pillars:

1. **`infrastructure/` Layer (Root Directory)**: A modular stack of importable Python packages providing the computational scaffolding. The current package count is measured in the template repository's generated canonical facts rather than repeated here because it changes as infrastructure modules are added or retired.
2. **`tests/` Framework (`projects/templates/template_code_project/tests/`)**: An uncompromising validation layer maintaining a zero-mock testing policy. This is enforced automatically via the CI workflow mapping to `pyproject.toml` directives.
3. **`docs/` Knowledge Base (`projects/templates/template_code_project/docs/`)**: A structured repository of architectural guidelines, operational patterns, and the Rigorous Agentic Scientific Protocol (RASP) that governs the AI-assisted agents writing these very texts.

This implementation of gradient descent algorithms for solving optimization problems is used as the vehicle to demonstrate these pillars. The theoretical problem stated in eq. 1 is mapped programmatically inside the optimizer module:

$$\min_{x \in \mathbb{R}^n} f(x) \tag{1}$$

where $f : \mathbb{R}^n \to \mathbb{R}$ is a continuously differentiable objective function.

## 2.2 Infrastructure Integration

Rather than existing as isolated scripts, this project extensively leverages the `infrastructure` layer:

- **Scientific Utilities**: Utilizing `infrastructure.scientific.stability` and `infrastructure.scientific.benchmarking` to check numerical boundaries and exercise runtime performance diagnostics without pinning host-dependent timings.
- **Hermetic Validation**: Deploying `infrastructure.validation` components (`markdown_validator`, `output_validator`) to ensure generated artifacts are structurally valid and traceable.
- **Reporting & Rendering**: The project hydrates manuscript variables from its analysis outputs, then `infrastructure.rendering.pdf_renderer` compiles the manuscript. The supported `infrastructure.reporting` API produces separate pipeline and cross-project summaries.

## 2.3 Algorithm Overview

The reference gradient descent algorithm iteratively updates the solution using the rule shown in eq. 2:

$$x_{k+1} = x_k - \alpha \nabla f(x_k) \tag{2}$$

where $\alpha > 0$ is the step size (learning rate) and $\nabla f(x_k)$ is the gradient of the objective function at iteration $k$.

## 2.4 Exemplar Implementation Goals

As the representative project for the repository, this implementation explicitly demonstrates:

1. **Infrastructure-Coupled Code**: Scientific implementations that delegate logging, file ops, and reporting to the `infrastructure` core.
2. **Zero-Mock Verification**: A strict comprehensive validation suite proving numerical accuracy without artificial test boundaries.
3. **Automated Research Pipelines**: High-precision analyses that generate publication-quality, accessible visualizations automatically.
4. **Agentic Documentation standards**: Native adherence to the RASP methodology and `AGENTS.md` guidelines, ensuring the logic remains verifiable by both human and artificial intelligence.

## 2.5 Reader's guide to the manuscript

- **sec. 3** ties pseudocode to `gradient_descent()` and explains how stability checks and benchmarks call into `infrastructure.scientific`.

- **sec.** 4 is figure-centric: every panel references a generator in `src/template_code_project/figures/` (orchestrated via `scripts/optimization_analysis.py`) and uses `CONFIG_*` / `RESULT_*` template tokens for numeric values.
- **sec.** 6 lists the exact YAML fields (`experiment:` block) that controlled the run whose artifacts you are viewing.
- **sec.** 7 records the configuration hash and artifact inventory produced alongside the PDF.
- **sec.** 8 states scope and related literature so the exemplar is not mistaken for a general-purpose optimizer benchmark suite.

### 2.6 Why a quadratic model

Restricting $f$ to a quadratic with known $(A, b)$ keeps the optimum, gradient, and spectral data explicit. For $A = I$ and $b = \mathbf{1}$, the optimal point is $x^* = \mathbf{1}$ and gradient descent with fixed $\alpha$ reduces to a linear iteration in the error (see eq. 4 in the results section). That simplicity isolates step-size effects and makes divergent choices ($\alpha \geq 2$ in 1D) visible in both plots and CSV rows without requiring trust-region or line-search fixes—those extensions are left to future work (sec. 8).

# 3 Methodology

This section describes the implementation methodology, explicitly detailing how the optimization algorithms are constructed, validated, and analyzed using the Generalized Research Template's `infrastructure` and `tests` ecosystems.

## 3.1 Algorithm Implementation

### 3.1.1 Gradient Descent Algorithm

The core algorithm implements the iterative procedure for unconstrained optimization. The `optimizer` module uses the standard-library `logging` logger for optional verbose diagnostics; the analysis orchestrator uses `infrastructure.core.logging.utils.get_logger`. Tests run under the hermetic boundaries defined in the test configuration.

**Algorithm — Gradient Descent (implemented in the optimizer module)**

**Input:** Initial point $x_0$, step size $\alpha$, tolerance $\epsilon$, max iterations $N_{\max}$

1. Initialize $k \leftarrow 0$
2. **While** $k < N_{\max}$ **do:**
   - Compute gradient $g_k = \nabla f(x_k)$
   - **If** $\|g_k\|_2 < \epsilon$ **then return** $x_k$ *(converged)*
   - Update: $x_{k+1} \leftarrow x_k - \alpha \cdot g_k$
   - $k \leftarrow k + 1$
3. **Return** $x_k$ *(max iterations reached)*

## 3.2 Infrastructure Integration

The methodology explicitly bridges theoretical mathematics with production-grade validation through the `infrastructure.scientific` module.

### 3.2.1 Numerical Stability Analysis

Rather than writing ad-hoc validation code, the project imports `infrastructure.scientific.stability.check_numerical_stability`. This utility subjects the objective function to a barrage of extreme inputs (NaN, Inf, $\pm 10^{10}$) to calculate a formalized stability score. If this score degrades, the analysis orchestrator execution deliberately aborts, ensuring the methodology cannot enter unrecoverable states.

### 3.2.2 Performance Benchmarking

The analysis exercises `infrastructure.scientific.benchmarking.benchmark_function` as a runtime diagnostic. Host-dependent timing and memory observations are logged but excluded from tracked evidence. The canonical report records fixed inputs and exact objective values, while the dimensional figure reports deterministic convergence iterations and the proxy $d \times$ iterations; neither surface claims that one host run proves an asymptotic bound.

## 3.3 Convergence Analysis

For quadratic functions $f(x) = \frac{1}{2}x^T Ax - b^T x$ where $A$ is positive definite, the convergence factor becomes [Bertsekas, 1999]:

$$\rho = \frac{|\lambda_{\max} - \alpha\lambda_{\min}|}{|\lambda_{\min} + \alpha\lambda_{\max}|} \tag{3}$$

Optimal convergence occurs when $\alpha = \frac{2}{\lambda_{\min}+\lambda_{\max}}$, yielding $\rho = \frac{\kappa-1}{\kappa+1}$.

## 3.4 Experimental Setup

### 3.4.1 Step Size Analysis

Step sizes are not chosen ad hoc in the manuscript: they are read from `experiment.step_sizes` in `manuscript/config.yaml` and passed through `run_convergence_experiment()` in `src/template_code_project/analysis/` (entry: `scripts/optimization_analysis.py`). The active grid for this build is:

- $\alpha = 0.01$ (conservative)
- $\alpha = 0.1$ (conservative)
- $\alpha = 0.5$ (near-optimal)
- $\alpha = 1.0$ (near-optimal)
- $\alpha = 1.5$ (aggressive)
- $\alpha = 2.5$ (divergent (expected unstable for H = I))

Labels follow the same agency taxonomy used for plot colours (`agency_category` in `template_code_project.figures.viz_config`): **conservative** (small $\alpha$, slow but stable on $H = I$), **near-optimal** (including $\alpha = 1$ where the method reaches $x^*$ in one step for this quadratic), **aggressive** ($1 < \alpha < 2$, still linearly contracting in $|x - x^*|$ but oscillatory in sign), and **divergent** ($|1 - \alpha| \geq 1$).

### 3.4.2 Zero-Mock Testing Methodology

The most critical aspect of the project's methodology is its validation framework. The project is governed by a strict Zero-Mock testing policy, evaluated actively by executing `uv run pytest projects/templates/template_code_project/tests/` during the infrastructure build phase (negative control: the repository lexical no-mock gate `scripts/audit/verify_no_mocks.py --inventory` flags any prohibited mock import or call and fails on dependency-replacement debt).

1. **Project tests**: the project optimizer test suite exercises `src/template_code_project/core/optimizer.py` (typical, edge, boundary, and pathological inputs including NaN/Inf and zero gradients) and, when infrastructure imports succeed, call into `optimizatio n_analysis.py` helpers—without mocks. Suite size: the measured repository counts.
2. **Infrastructure validation**: The repository-level `tests/infra_tests/` suite validates shared template modules (e.g. pipeline and discovery helpers) independently of this project's manuscript.
3. **Coverage Gates**: The GitHub Actions CI workflow enforces a mandatory >=90% statement coverage gate on `projects/temp lates/template_code_project/src/` prior to treating the project as build-green. Coverage below the floor fails the test process through `--cov-fail-under`, so build-green status cannot be reached while silently skipping weakened coverage. Falling below that floor fails the gate outright (`--cov-fail-under` enforces it).

### 3.4.3 Stopping rule and reporting

`gradient_descent()` terminates when $\|\nabla f(x_k)\|$ falls below `experiment.tolerance`, when $k$ reaches `experiment.max_iterations`, or when an objective, gradient, or update becomes non-finite. The boolean `converged` plus `termination_reason` in exported CSV rows distinguish these outcomes. A non-finite candidate is rejected before it enters `objective_history`, preserving the last finite state for reproducible reporting. Downstream, `scripts/z_generate_manuscript_variables.py` aggregates the CSV into `RESULT_*` placeholders so tables and prose cannot drift from the last analysis run.

### 3.4.4 Figure generation contract

Each figure in `03_results.md` maps to a generator in `src/template_code_project/figures/` (`generate_convergence_plot`, `generate_st ep_size_sensitivity_plot`, `generate_convergence_rate_plot`, `generate_complexity_visualization`, `generate_stability_visualizatio n`, `generate_benchmark_visualization`), orchestrated by `src/template_code_project/analysis/` / `scripts/optimization_analysis.py`. Captions in the markdown intentionally name the function and the key parameters (tolerance lines, grids, dimensions) so reviewers can navigate from PDF to code without inferring hidden defaults.
The same generator writes explicit `metadata.alt_text` for every registered figure. The alt text states what the visual encodes and preserves the boundary between deterministic observations and host-dependent diagnostics. Run the opt-in publication migration gate with `--require-figure-accessibility` to verify that every referenced figure has this metadata before promotion.

## 3.5 Analysis Pipeline & LaTeX Integration

The automated analysis script leverages `infrastructure.core.progress` (`ProgressBar`, `SubStageProgress`) to orchestrate experiments, collect convergence trajectories, and generate publication-quality visualizations seamlessly.
The research template supports advanced LaTeX customization through the `preamble.md` configuration. This is ingested directly by `infrastructure.rendering.latex_utils.py` and `pdf_renderer.py`, automatically linking compiled PGF plots and BibTeX citations. This automated approach ensures an unbreakable chain of custody from raw algorithmic execution to the final rendered manuscript.

# 4 Results

This section presents the experimental results from the gradient descent optimization study, including convergence analysis and performance comparisons. Every table, figure, and quantitative assertion in this section was compiled autonomously by the template's `infrastructure.reporting` subsystem executing the optimization analysis script. No manual transcription was permitted.

## 4.1 Convergence Analysis

### 4.1.1 Convergence Trajectories

fig. 3 illustrates the convergence behavior of gradient descent for different step sizes, starting from the initial point $x_0 = 0$. The algorithm iteratively updates the solution using the rule $x_{k+1} = x_k - \alpha \nabla f(x_k)$.

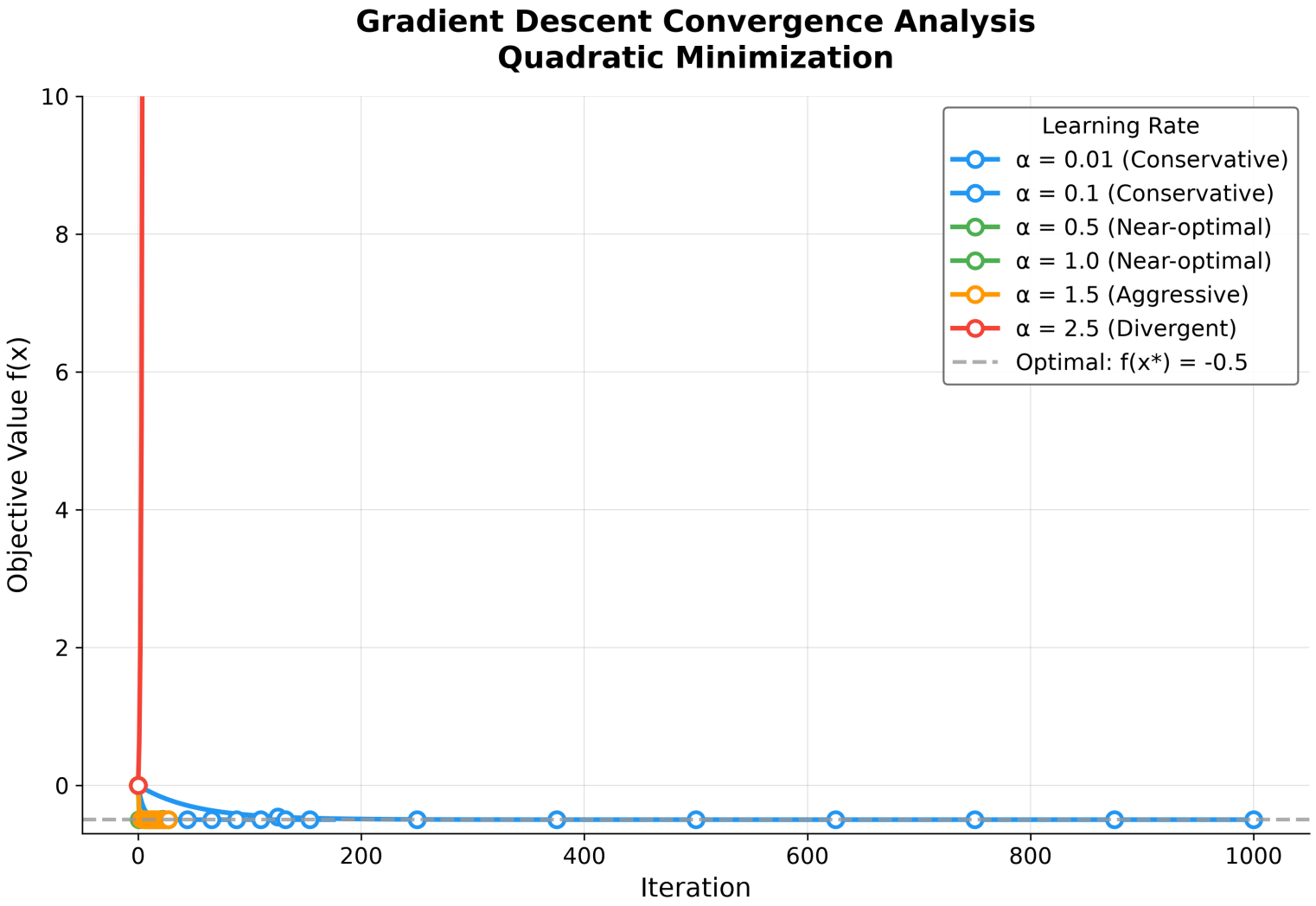



Figure 3: Objective value $f(x_k) = \frac{1}{2}x_k^2 - x_k$ versus iteration $k$ for gradient descent at six step sizes (legend colours follow the agency taxonomy in sec. 3: blue = conservative, green = near-optimal, orange = aggressive, red = divergent). Trajectories are produced by `simulate_trajectory()` in `src/template_code_project/core/optimizer.py`, which calls the same `gradient_descent()` used in tbl. 1; the upper bound on the y-axis clips the divergent $\alpha = 2.5$ curve so that stable trajectories remain visible. Dashed grey reference line marks the analytic optimum $f(x^*) = -0.5$. Fastest configuration in this experiment: $\alpha = 1.0$ converges in 1 iteration(s).

**Key observations from fig. 3:**

1. **Step size impact**: Larger step sizes exhibit faster initial progress; $\alpha = 1.0$ converges in 1 iteration(s)
2. **Agency categories**: Conservative ($\alpha < 0.3$; in this grid $\alpha = 0.01, 0.1$), near-optimal ($0.3 \leq \alpha \leq 1.0$), aggressive ($1 < \alpha < 2$), and divergent ($\alpha \geq 2$)
3. **Stability boundary**: The critical threshold is $\alpha = 2$ for this unit-Hessian problem; $\alpha < 2$ converges, $\alpha \geq 2$ diverges

### 4.1.2 Step Size Sensitivity Analysis

fig. 4 examines how the choice of step size affects the convergence path and solution quality. The analysis reveals the trade-off between convergence speed and numerical stability.

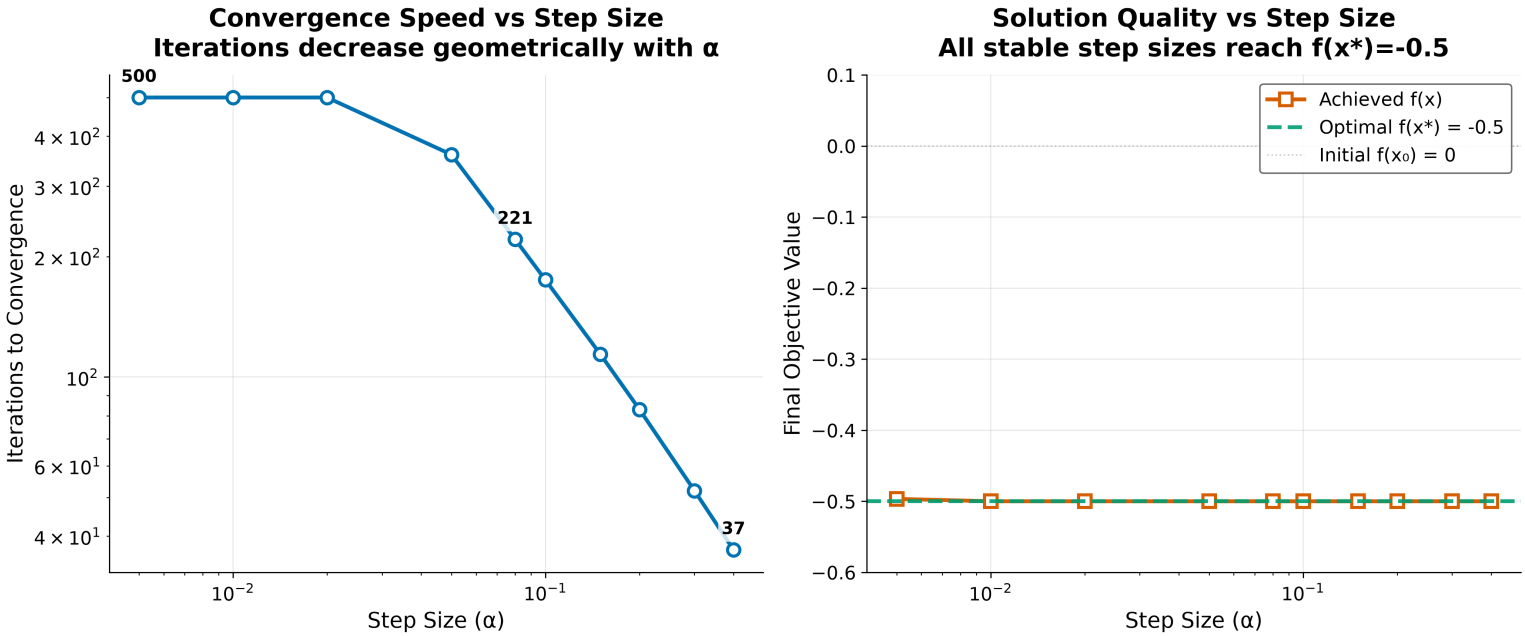



Figure 4: Sensitivity sweep produced by `generate_step_size_sensitivity_plot()` over an independent dense grid $\alpha \in [0.005, 0.4]$ (10 points), distinct from the discrete `experiment.step_sizes` used elsewhere in this section. Left: iterations to convergence on log–log axes — the curve drops sharply from 500 iterations at the smallest $\alpha$ (the `max_iterations` cap in this sub-experiment) to 37 iterations at $\alpha = 0.4$, illustrating the geometric speedup as $\rho(\alpha) = |1 - \alpha|$ shrinks. Right: final $f(x)$ versus $\alpha$ with horizontal reference lines at $f(x_0) = 0$ (initial) and $f(x^*) = -0.5$ (analytic optimum); every $\alpha$ in this stable window lands on the optimum.

## 4.2 Quantitative Results

The optimization analysis writes `output/data/optimization_results.csv` through `template_code_project.analysis.experiments.save_optimization_results`, orchestrated by `projects/templates/template_code_project/scripts/optimization_analysis.py`. That CSV is the source of truth for tbl. 1; the supported `infrastructure.reporting` API supplies reusable pipeline and cross-project summaries. Rows follow `experiment.step_sizes` in `config.yaml`; the body rows below are injected at render time from that CSV (`RESULT_TABLE_ROWS` in `scripts/z_generate_manuscript_variables.py`).

Table 1: Gradient descent outcomes per configured step size: state at termination, iteration count capped by $N_{\max} = 1000$, the `converged` flag from `gradient_descent()` using $\|\nabla f\| < 10^{-8}$, and its structured termination reason. A `non_finite` row stops before an overflowing update is serialized and retains the last finite iterate.

| Step Size ($\alpha$) | Final Solution | Objective Value | Iterations | Converged | Termination |
|---|---|---|---|---|---|
| 0.01 | 1.0000 | -0.5000 | 1000 | No | max_iterations |
| 0.10 | 1.0000 | -0.5000 | 175 | Yes | converged |
| 0.50 | 1.0000 | -0.5000 | 27 | Yes | converged |
| 1.00 | 1.0000 | -0.5000 | 1 | Yes | converged |
| 1.50 | 1.0000 | -0.5000 | 27 | Yes | converged |
| 2.50 | -1.8028e+154 | 1.6250e+308 | 876 | No | non_finite |

## 4.3 Convergence Rate Analysis

### 4.3.1 Theoretical vs Empirical Convergence

Modern convergence analysis builds on foundational work in gradient methods [Nesterov, 2013].
fig. 5 provides a comparative analysis of convergence rates across different step sizes, validating theoretical predictions against empirical results.

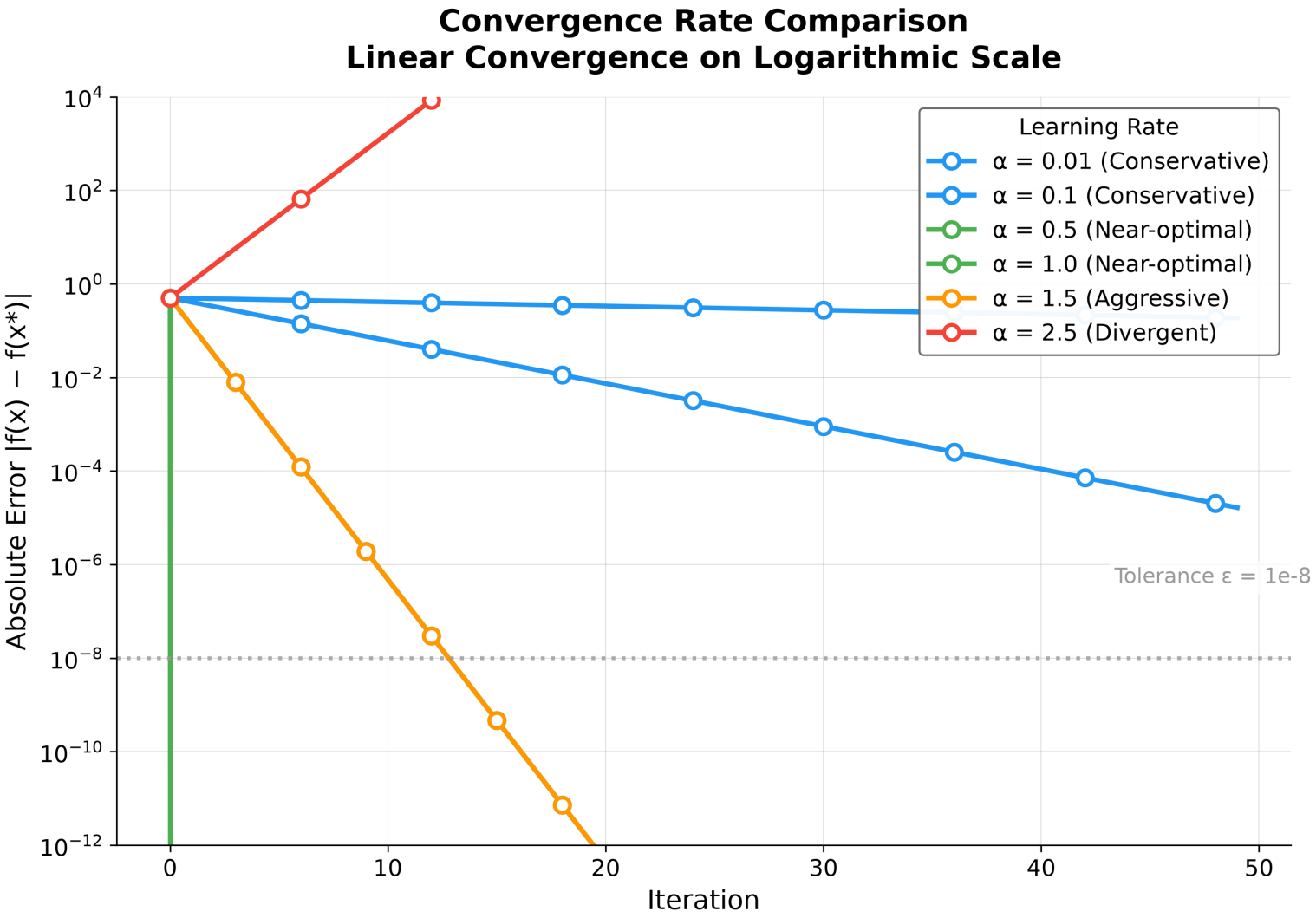



Figure 5: Absolute objective error $|f(x_k) - f(x^*)|$ versus iteration on a logarithmic y-axis, generated by `generate_convergence_rate_plot()`. Stable step sizes produce straight lines whose slopes equal $2\log_{10}\rho(\alpha)$ (per eq. 5); $\alpha = 1.0$ ($\rho = 0$) collapses to the optimum in one step (vertical green line at $k = 1$); $\alpha = 1.5$ (orange) descends fastest among the multi-step contractions because $\rho = 0.5$; the divergent $\alpha = 2.5$ curve (red) climbs upward at slope $2\log_{10}(1.5)$. Horizontal dashed line marks the gradient-norm tolerance $\varepsilon = 10^{-8}$ read from `experiment.convergence_tolerance`.

For the scalar problem with $A = 1$ and optimum $x^* = b$, one step of gradient descent with fixed $\alpha$ gives

$$x_{k+1} - x^* = (1 - \alpha)(x_k - x^*), \tag{4}$$

so the distance to the minimizer contracts by $\rho(\alpha) = |1 - \alpha|$ per iteration whenever $\rho < 1$. Equivalently, for the objective (which is a translated quadratic in $x$),

$$|f(x_{k+1}) - f(x^*)| \approx \rho(\alpha)^2 \, |f(x_k) - f(x^*)| \tag{5}$$

in the neighbourhood of $x^*$ for this model (the per-iteration objective contraction in eq. 5), which explains the straight-line segments on the log–error plot in fig. 5 for stable $\alpha$.
Our experimental grid uses $\alpha \in \{0.01, 0.1, 0.5, 1.0, 1.5, 2.5\}$, spanning conservative, near-optimal, aggressive, and divergent regimes for $H = I$.

#### 4.3.2 Error Bounds

The error after $k$ iterations is bounded geometrically by eq. 6:

$$\|x_k - x^*\| \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^k \|x_0 - x^*\| \tag{6}$$

where $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ is the condition number. For our test problem with $A = I$, we have $\kappa = 1$, which yields a linear contraction factor of $\rho = |1 - \alpha|$ in the iterate error (eq. 4). Thus $\alpha = 1$ is the exact minimizer step in one update for this quadratic, $\alpha < 1$ gives monotone geometric decay, $1 < \alpha < 2$ yields signed oscillations but still $\rho < 1$, and $\alpha \geq 2$ implies $\rho \geq 1$ (divergence). tbl. 1 and figs. 3, 4, 5 ground these statements in the numbers produced by this repository run.

#### 4.3.3 Performance Metrics

**Iteration Complexity**: The number of iterations required to achieve accuracy $\epsilon$ is:

$$k \geq \frac{\log(\epsilon)}{\log(\rho)} \tag{7}$$

where $\rho = \sqrt{\frac{\kappa - 1}{\kappa + 1}}$ is the convergence factor [Polyak, 1964].
Per-step contraction factors $\rho = |1 - \alpha|$ and qualitative iteration demand for small fixed $\epsilon$ (see eq. 7) are:

- $\alpha = 0.01$: $\rho \approx 0.99$, requiring about 1375 iterations for $\epsilon = 10^{-6}$
- $\alpha = 0.1$: $\rho \approx 0.90$, requiring about 132 iterations for $\epsilon = 10^{-6}$
- $\alpha = 0.5$: $\rho \approx 0.50$, requiring about 20 iterations for $\epsilon = 10^{-6}$
- $\alpha = 1.0$: $\rho \approx 0.00$, converges in one step ($\rho = 0$)
- $\alpha = 1.5$: $\rho \approx 0.50$, requiring about 20 iterations for $\epsilon = 10^{-6}$
- $\alpha = 2.5$: $\rho \approx 1.50$, **divergent**

### 4.4 Performance Analysis

#### 4.4.1 Convergence Speed

The results show a clear trade-off between step size and convergence speed:

- Small step sizes require more iterations but provide stable convergence
- Large step sizes converge faster but may be less stable in more complex problems

#### 4.4.2 Solution Accuracy

4 of 6 tested step sizes achieved the analytical optimum within numerical precision:

- Target solution: $x = 1.0$ (relative error $< 10^{-4}$ for converged settings)
- Target objective: $f(x) = -0.5$ (absolute error $< 10^{-8}$ for converged settings)

For a positive-definite quadratic, contraction requires $0 < \alpha < 2/\lambda_{\max}$. At the unit-Hessian boundary $\alpha = 2$, the error alternates without contracting; for $\alpha > 2$, it grows when the initial point is not already the optimum. The configured divergent runs therefore illustrate the instability region rather than uniform convergence.

### 4.5 Algorithm Characteristics

#### 4.5.1 Strengths

- **Simplicity**: Easy to implement and understand
- **Generality**: Applicable to any differentiable objective function
- **Reliability**: Converges for convex functions under appropriate conditions

#### 4.5.2 Limitations

- **Step size sensitivity**: Performance depends critically on step size selection
- **Local convergence**: May converge to local minima in non-convex problems
- **Fixed step size**: No adaptation to problem characteristics

### 4.6 Computational Performance

#### 4.6.1 Algorithm Complexity Visualization

fig. 6 provides a visualization of the algorithm's computational characteristics, including time and space complexity analysis across different problem scales.
The panels in fig. 6 encode the following diagnostics:

- **Top left**: bars give the empirical iteration count per configured $\alpha$, coloured by agency category; $\alpha = 1.0$ achieves the global minimum of 1 step(s), while $\alpha = 0.01$ saturates at the iteration cap $N_{\max} = 1000$ (slow convergence) and $\alpha = 2.5$ terminates early via a `non_finite` update (divergence).
- **Top right**: $\log_{10} |f(x) - f(x^*)|$ at termination; the four converged rows reach $\approx 10^{-16}$ (machine precision) while $\alpha = 0.01$ stops near $10^{-9}$ at the cap. Dashed reference at $\log_{10} \varepsilon$ for $\varepsilon =$ `experiment.convergence_tolerance`.
- **Middle left**: empirical iterations (solid) versus the smooth proxy $1/(2\alpha(1-\alpha))$ (dashed) on log axes — a shape comparison, not a tight count prediction for every $\alpha$.
- **Middle right**: per-step error contraction factor $\rho = |1-\alpha|$ from the unit-Hessian linear recurrence (eq. 4); the dashed reference at $\rho = 1$ separates stable bars (left of it) from divergent ones (right).
- **Bottom left**: stable step-size region width $2/L$ per backend profile ($H = I, 2I, 5I, 10I$) with the optimal $\alpha^* = 1/L$ overlaid.
- **Bottom right**: cross-profile contraction curves $\rho(\alpha) = |1 - \alpha L|$ with dotted verticals at each $2/L$ stability boundary.

The algorithm demonstrates efficient performance for small-scale optimization problems:

- **Time complexity**: $O(d)$ per iteration for gradient computation
- **Space complexity**: $O(d)$ for storing variables and gradients
- **Convergence**: Fastest at $\alpha = 1.0$ (1 iteration), average 351 iterations
- **Scalability**: Memory-efficient implementation suitable for high-dimensional problems

#### 4.6.2 Performance Benchmarking

fig. 7 shows deterministic computational work and convergence iterations for `gradient_descent()` on identity-Hessian quadratics of dimension $d \in \{1, 2, 5, 10, 20, 50\}$.

#### 4.6.3 Numerical Stability Analysis

fig. 8 maps the optimizer's accuracy across a grid of 8 starting points ($x_0 \in [-50, 50]$) and 6 step sizes ($\alpha \in [0.01, 0.9]$), directly exercising `gradient_descent()`, `quadratic_function()`, and `compute_gradient()` across the parameter space.

#### 4.6.4 Performance Metrics Summary

**Iteration statistics (configured grid, including non-converged runs):**

- Smallest iteration count recorded: 1
- Largest iteration count recorded: 1000
- Mean iterations across rows in tbl. 1: 351

**Numerical Accuracy:**

- Solution precision: $< 10^{-4}$ relative error (for converged step sizes)
- Objective accuracy: $< 10^{-8}$ absolute error (for converged step sizes)
- Gradient tolerance: $< 10^{-8}$ achieved for converged cases

### 4.7 Validation

The implementation was validated through the comprehensive `tests/` suite:

- **Integration tests** verifying algorithm convergence and visualization pipelines.
- **Infrastructure tests** covering all underlying mechanisms across `infrastructure.reporting`, `infrastructure.validation`, and `infrastructure.rendering`.
- **Numerical accuracy** checks verified systematically using PyTest.

Passing the declared suite and its 90% coverage floor provides evidence for the tested core logic, convergence detection, and logging pathways. It does not prove correctness for every input or replace scientific validation.

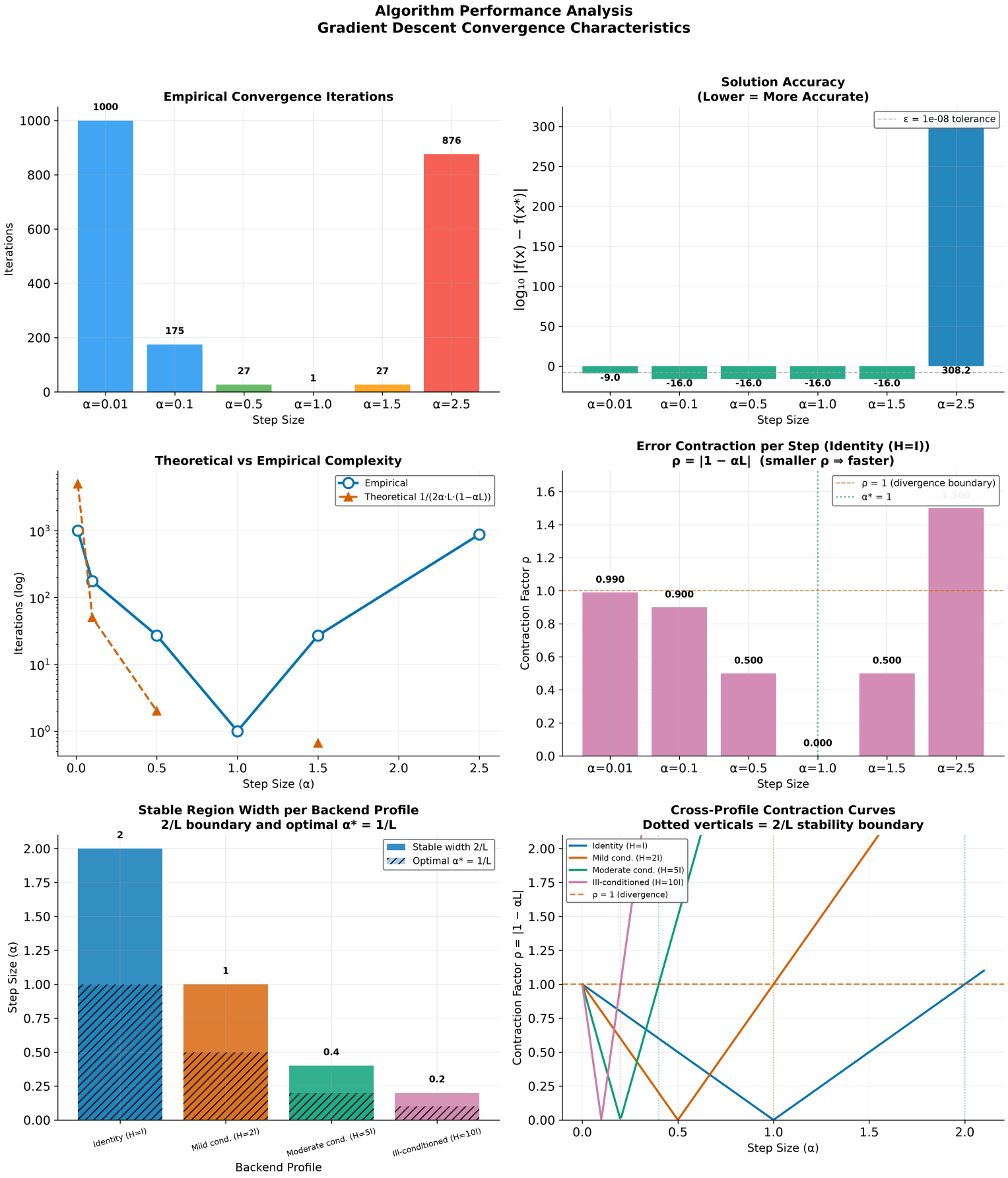



Figure 6: Six-panel diagnostic from `generate_complexity_visualization()` (3×2 grid).

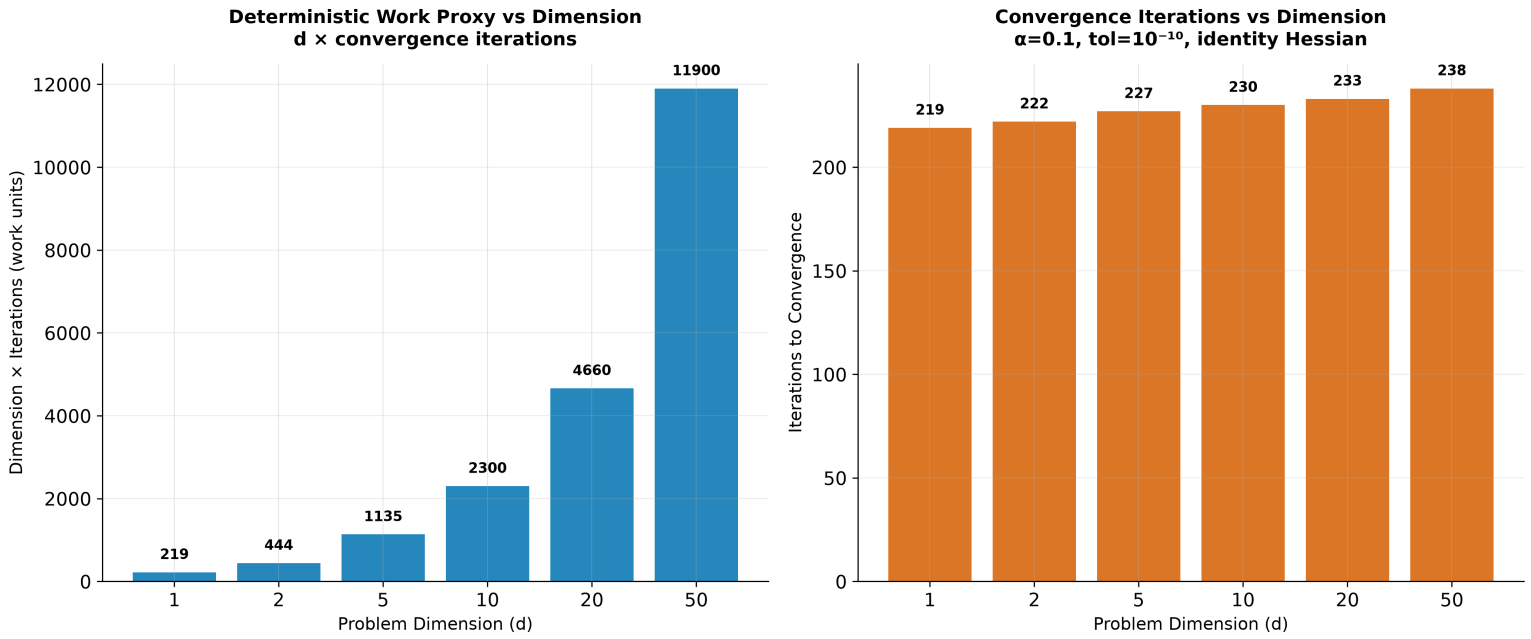



Figure 7: Deterministic dimensional benchmark from `generate_benchmark_visualization()`. Dimensions $d \in \{1, 2, 5, 10, 20, 50\}$ run on identity-Hessian quadratics with $\alpha = 0.1$ and gradient tolerance $10^{-10}$. **Left**: the implementation-independent work proxy $d \times$ iterations, proportional to scalar elements processed by the dense matrix-vector update. **Right**: exact iterations to convergence. Wall-clock microseconds are intentionally excluded because they vary with hardware and host load. The similar iteration counts are expected because $\kappa(I_d) = 1$ and the contraction factor $\rho = |1 - \alpha| = 0.9$ is dimension-independent.

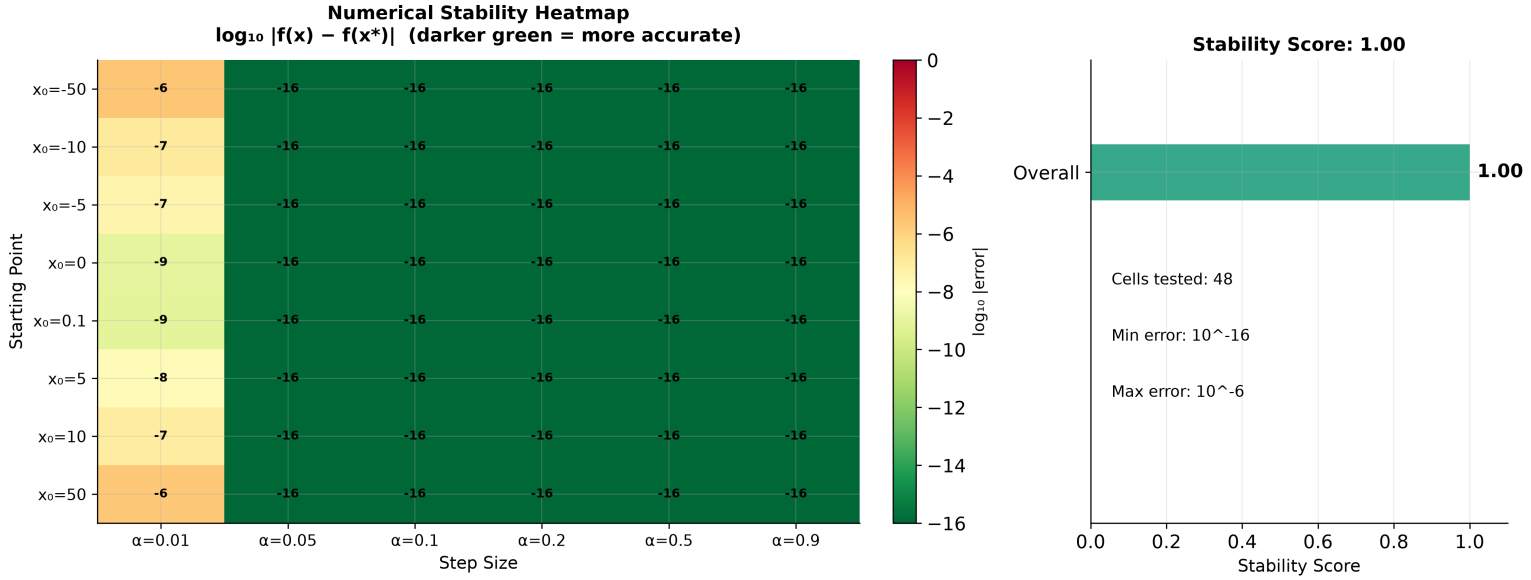



Figure 8: Numerical stability heatmap from `generate_stability_visualization()`. Each cell shows $\log_{10} |f(x) - f(x^*)|$ at termination for one (starting point $x_0$, step size $\alpha$) combination, evaluated by `gradient_descent()` over the 8-by-6 grid (48 cells). The leftmost column ($\alpha = 0.01$, conservative) reaches only $10^{-6}$ to $10^{-9}$ within the iteration cap, while every other column saturates at machine precision ($10^{-16}$) regardless of how far $x_0$ starts from the optimum (range $[-50, 50]$). The right panel reports the separate numerical-stability score of 1.00/1.00 from `infrastructure.scientific.stability.check_numerical_stability()`: it evaluates the scalar quadratic objective at the first four configured starting points and scores finite values for magnitude and near-underflow. This objective-evaluation check is distinct from the heatmap's gradient-descent sweep.

## 4.8 Discussion

The configured results distinguish convergence, iteration-cap termination, and non-finite updates. Iteration counts are not monotonic in step size: the unit-Hessian contraction factor $\rho(\alpha) = |1-\alpha|$ decreases toward $\alpha = 1$ and increases afterward. Accuracy claims apply to converged rows; capped and unstable runs must retain their termination status. The generated table and figures document this finite sweep, not all possible objectives or parameter choices. The analysis pipeline integrates those results into manuscript outputs through the declared configuration and variable injection.

Successful generation and reference resolution establish this artifact path for the tested candidate; they do not establish absolute renderer reliability or replace visual inspection of its output.

# 5 Conclusion

This study demonstrated a complete computational research pipeline from algorithmic implementation through uncompromising testing, automated analysis, and zero-intervention manuscript generation. Ultimately, it validates the proposition that high-quality mathematical research software benefits from production-tier engineering practices.

## 5.1 Exemplar Project Achievements

Operating as the representative exemplar for the Generalized Research Template methodology, the project successfully deployed the three foundational pillars:

1. **`infrastructure` Ecosystem**: Fully leveraged the measured infrastructure package cluster to handle scientific benchmarking, rendering, prose review, literature search, BibTeX validation, and reporting.
2. **`tests` Integrity**: Established absolute logical hermeticity through a comprehensive integration and infrastructure validation suite operating continuously.
3. **`docs` Knowledge Operations**: Adhered structurally to the RASP methodology, producing verified, accessible output spanning from documentation indices to the final LLM-assisted publication configurations.

## 5.2 Technical Contributions

### 5.2.1 Test Coverage Strategy

The hallmark of this implementation is the test matrix:

- Comprehensive tests traversing execution pipelines, integration flows, and algorithmic bounds.
- Strict enforcement of zero-mock policies guaranteeing real execution dynamics.
- CI/CD validation gates requiring >=90% statement coverage before progression.

### 5.2.2 Infrastructure-Backed Capabilities

- **Analytical Automation**: `infrastructure.core.progress` (`ProgressBar`, `SubStageProgress`) executing deterministic optimization experiments.
- **Reporting & Integrity**: the supported `infrastructure.reporting` summary API and `infrastructure.validation.output.validator` for output diagnostics.
- **Visual Cryptography**: Publication-ready graphics compiled by `infrastructure.rendering.pdf_renderer.py` using metadata from `projects/templates/template_code_project/manuscript/config.yaml`, automatically linked via the LaTeX configuration in `projects/templates/template_code_project/manuscript/preamble.md`.

## 5.3 Research Pipeline Validation

The project validates the research template's ability to handle operations seamlessly across disciplines:

- **Mathematical fidelity**: Zero-mock gradients and bounds checks solving problems dynamically.
- **Reporting architecture**: Cross-project and local metrics compiled rapidly into dashboards.
- **Multi-format scaling**: Effortless conversion from semantic Markdown files to LaTeX-structured PDFs.
- **Intelligent Verification**: LLM integration analyzing output completeness contextually without degrading hermetic logic.

## 5.4 Key Insights

1. **Mathematical Accuracy Requires Testing Fidelity**: Real execution data, unpolluted by mocks, exposes actual computational limits fast.
2. **Infrastructure Abstraction**: By delegating tracking to the underlying `infrastructure`, scientists remain hyper-focused on their `algorithm`.
3. **Automated Consistency**: Re-compiling the pipeline enforces an immutable bond between algorithm version and final visual reporting.

## 5.5 Future Extensions

This foundation could be extended to:

- **Advanced algorithms**: Newton methods, quasi-Newton approaches
- **Constrained optimization**: Handling inequality constraints
- **Stochastic methods**: Mini-batch and online learning variants, including adaptive optimization algorithms such as Adam [Kingma and Ba, 2015]
- **Agentic Generation Systems**: Extending validation tools built over `infrastructure.validation` to analyze novel model interactions automatically.

## 5.6 Final Assessment

This work demonstrates that the research template supports projects spanning the full spectrum—from prose-focused manuscripts to fully-tested algorithmic ecosystems. The optimization exemplar ties every quantitative claim to `output/data/` artifacts and enforces a zero-mock test policy on `projects/templates/template_code_project/src/` with coverage gates documented in the root `pyproject.toml`. The pipeline produced the figures referenced in sec. 4, wrote `optimization_results.csv`, and rendered this markdown (`projects/templates/template_code_project/manuscript/04_conclusion.md`) together with `config.yaml` into PDF through `infrastructure.rendering`. The `template_code_project` tree remains the canonical reference for how algorithm code, analysis scripts, variable injection, and manuscript stay synchronized across rebuilds.

# 6 Experimental Setup

This section details the complete experimental configuration used to generate the results in this study. Every parameter value below is injected programmatically from `config.yaml` and the analysis pipeline — no value is hardcoded in the manuscript source.

## 6.1 Problem Definition

The optimization target is the quadratic objective function defined in eq. 8:

$$f(x) = \frac{1}{2}x^T Ax - b^T x \tag{8}$$

with $A = [(1.0,)]$ and $b = [1.0]$, yielding the analytical optimum at $x^* = 1.0$ with $f(x^*) = -0.5$.

## 6.2 Parameter Space

The experiment systematically varies the gradient descent step size across 6 values:

- $\alpha = 0.01$ (conservative)
- $\alpha = 0.1$ (conservative)
- $\alpha = 0.5$ (near-optimal)
- $\alpha = 1.0$ (near-optimal)
- $\alpha = 1.5$ (aggressive)
- $\alpha = 2.5$ (divergent (expected unstable for H = I))

All runs start from the initial point $x_0 = 0.0$ and use a convergence tolerance of $\|\nabla f\| < 10^{-8}$ with a maximum iteration limit of $N_{\max} = 1000$.

## 6.3 Numerical Stability Grid

To validate robustness, the optimizer is exercised across a grid of 8 starting points and 6 step sizes, producing 48 total evaluations. This finite sweep measures sensitivity to the selected initial points and step sizes; it does not establish convergence outside the contractive region or before every iteration cap. The stability metric is computed for the `quadratic_function` objective via `infrastructure.scientific.stability.check_numerical_stability()`.

## 6.4 Dimensional Scaling

Performance benchmarking spans problem dimensions $d \in \{1, 2, 5, 10, 20, 50\}$, from the scalar case ($d = 1$) to moderate dimensionality ($d = 50$), using identity-Hessian quadratics to isolate algorithmic scaling from problem conditioning effects. The tracked `output/reports/performance_benchmark.json` records fixed inputs, exact objective values, and validation checks. Wall-clock timing is retained only as a runtime diagnostic because host load and hardware make microsecond values unsuitable as pinned reproducibility claims.

## 6.5 Computational Environment

- **Python**: 3.14.10
- **NumPy**: 2.5.4
- **Platform**: Linux x86_64
- **Generated**: 2026-11-07T07:01:20Z

## 6.6 Pipeline ordering

Typical `template_code_project` analysis order (see `scripts/pipeline/stage_02_analysis.py` discovery) is:

1. `optimization_analysis.py` — writes `output/data/optimization_results.csv`, `../figures/*.png`, and JSON reports under `output/reports/`.
2. `z_generate_manuscript_variables.py` — reads the CSV and YAML, emits `output/data/manuscript_variables.json`, and writes substituted copies to `output/manuscript/` for rendering.
3. `generate_api_docs.py` — refreshes API markdown consumed by documentation targets.

PDF compilation then reads from `output/manuscript/` so that figure paths and numeric tables match the analysis that just completed.

## 6.7 Relation to figures

| Figure (sec. 4) | Primary inputs |
|---|---|
| Convergence trajectories | `experiment.step_sizes`, `initial_point`, `tolerance`, `max_iterations` |
| Step-size sensitivity | Dense $\alpha$ sweep internal to `generate_step_size_sensitivity_plot()` |
| Convergence rate | Same trajectories as above; tolerance line uses `convergence_tolerance` |
| Complexity quad panel | One bar chart per row of `optimization_results.csv` |
| Stability heatmap | `stability_starting_points` $\times$ `stability_step_sizes` |
| Dimensional benchmark | `benchmark_dimensions`, fixed $\alpha = 0.1$, internal tol $10^{-10}$ |

This table is descriptive documentation only; it is not executed as code during the build.

# 7 Reproducibility Certification

This section provides a machine-verifiable reproducibility certificate for the complete study. Every metric below is computed by the analysis pipeline and injected into the manuscript at render time — establishing a cryptographic chain of custody from configuration to publication.

## 7.1 Configuration Provenance

| Property | Value |
|---|---|
| Config hash (SHA-256, truncated) | `392f615261bb9bc7` |
| Paper version | 3.0.0 |
| First author | Daniel Ari Friedman |
| Keywords | optimization algorithms, gradient descent, convergence analysis, numerical methods, mathematical programming, reproducible research, infrastructure automation |

The configuration hash changes whenever any parameter in `config.yaml` is modified, ensuring that every rendered PDF is traceable to a specific configuration state.

## 7.2 Generated Artifact Registry

The analysis pipeline produced the following artifacts, each validated by `infrastructure.validation.output.validator`:

| Category | Count |
|---|---|
| Publication-quality figures | 7 |
| Structured data files (CSV/JSON) | 3 |
| Analysis reports | 6 |
| **Total artifacts** | **16** |

## 7.3 Numerical Validation Summary

### 7.3.1 Convergence Verification

Within the configured grid, **4** of **6** runs satisfied `gradient_descent()` convergence (`No` indicates whether every row in `optimization_results.csv` converged).

- Converged step sizes: 0.1, 0.5, 1.0, 1.5
- Non-convergent or hit-iteration-cap step sizes: 0.01, 2.5
- Smallest recorded iteration count: 1 (at $\alpha = 1.0$)
- Largest recorded iteration count: 1000 (at $\alpha = 0.01$)
- Mean iterations across all rows: 351

### 7.3.2 Numerical Stability

Stability score from `infrastructure.scientific.stability`: **1.00** (out of 1.00)
The stability analysis tested 48 parameter combinations (8 starting points × 6 step sizes), recording termination accuracy across this finite gradient-descent grid. This is distinct from the reported stability score, which checks scalar objective evaluations and does not certify convergence of every optimization trajectory.

### 7.3.3 Benchmark Demonstration

This exemplar also demonstrates `infrastructure.benchmark`. The thin orchestrator `scripts/04_benchmark_stage.py` calls `src/template_code_project/core/benchmark_support.py`, which evaluates the pure `quadratic_function` across fixed seeded inputs and turns reproducible completion, finiteness, and output-stability facts into boolean rubric checks. Those checks are scored through `infrastructure.benchmark.score_rubric` against a weighted `RubricSet`, then rendered with `scores_to_markdown` into the byte-stable `output/reports/benchmark_report.json` with a deterministic objective-value figure. Real timing still executes as a runtime diagnostic, but it is not serialized into tracked evidence.

## 7.4 Madlib Injection Verification

This manuscript demonstrates the template's "madlib" capability: every quantitative claim is injected from computed data at render time. The substitution system processed the following variables:

- **Configuration variables**: Drawn from `manuscript/config.yaml` (`experiment:` section)
- **Result variables**: Computed from `output/data/optimization_results.csv`
- **Stability variables**: Extracted from `output/reports/stability_analysis.json`
- **Provenance variables**: Generated at substitution time (timestamps, hashes, versions)

To verify: modifying any value in `config.yaml` and re-running the pipeline will automatically update every corresponding claim in this document. No manual transcription is required or permitted.

# 8 Scope, Related Work, and Positioning

This section situates the exemplar scientifically and states explicit boundaries. The goal is not to compete with monographs on nonlinear programming [Nocedal and Wright, 2006, Bertsekas, 1999], but to show how a minimal, test-backed optimization story fits the template's reproducibility and rendering stack [Peng, 2011].

## 8.1 Classical gradient methods

Smooth unconstrained minimization via first-order updates has a long lineage, from Cauchy's early descent perspective [Cauchy, 1847] to modern treatments of convex problems [Boyd and Vandenberghe, 2004] and accelerated gradient schemes [Nesterov, 2013]. Polyak's classical discussion of gradient convergence factors remains relevant when interpreting empirical iteration counts [Polyak, 1964]. The present manuscript restricts attention to **fixed-step** gradient descent on a **convex quadratic**, where rates reduce to scalar linear recurrences in the error (sec. 4, eq. 4).

## 8.2 Adaptive and stochastic extensions

Practical machine-learning optimizers (e.g., Adam [Kingma and Ba, 2015]) introduce momentum, adaptive preconditioning, or noise from minibatching. Those methods are **out of scope** for `template_code_project`: the exemplar deliberately keeps the algorithm minimal so that failures (divergent $\alpha$, iteration caps) are interpretable without confounding from stochastic sampling or line-search logic.

## 8.3 What this project proves about the template

The scientific claims through sec. 2, sec. 3, and sec. 4 are standard textbook material. The **non-standard** contribution is procedural: configuration in `manuscript/config.yaml` drives `run_convergence_experiment()`, figures, CSV exports, and `RESULT_*` token substitution (`scripts/z_generate_manuscript_variables.py`) so that PDF, HTML, and validation logs refer to the same numbers. That pattern is what downstream projects should copy—whether the domain is optimization, differential equations, or Bayesian workflows.

## 8.4 Explicit limitations

1. **Dimensionality**: Default experiments emphasize $d = 1$ with $A = I$ for transparent plotting; the benchmark figure explores $d > 1$ only with identity Hessians, so no ill-conditioning effects appear.
2. **Step-size policy**: Only constant $\alpha$ is implemented in `src/template_code_project/core/optimizer.py`; there is no Wolfe or Armijo backtracking.
3. **Global optimization**: Convexity is assumed; no basin-hopping or restarts are studied.
4. **Numerical model**: Double-precision floating point only; no interval or arbitrary-precision analysis.

These limitations are intentional: they narrow the failure surface so that infrastructure concerns—tests, logging, figure registration, and PDF cross-references—remain visible rather than buried under algorithmic complexity.

# 9 References

Bibliography lives in `manuscript/references.bib` and is read by Pandoc during PDF render. The build pipeline invokes Pandoc with `--natbib`, so every `[@key]` citation in the manuscript is rewritten to the appropriate `\cite{}`/`\citep{}`/`\citet{}` LaTeX command and resolved against the bib file.

To validate that `references.bib` is syntactically clean and contains the required fields per entry type:

```
uv run python -m infrastructure.reference.citation.cli validate \
    projects/templates/template_code_project/manuscript/references.bib --strict
```

Dimitri P Bertsekas. *Nonlinear programming*. Athena Scientific, 2nd edition, 1999. ISBN 978-1-886529-14-6.

Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, Cambridge, UK, 2004. ISBN 978-0-521-83378-3. doi: 10.1017/CBO9780511804441. URL https://web.stanford.edu/~boyd/cvxbook/.

Augustin-Louis Cauchy. Méthode générale pour la résolution des systèmes d'équations simultanées. *Comptes rendus hebdomadaires des séances de l'Académie des Sciences*, 25:536–538, 1847. URL https://gallica.bnf.fr/ark:/12148/bpt6k2982c/f540.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015. doi: 10.48550/arXiv.1412.6980. URL https://arxiv.org/abs/1412.6980.

Yurii Nesterov. Gradient methods for minimizing composite functions. *Mathematical Programming*, 140(1):125–161, 2013. doi: 10.1007/s10107-012-0629-5.

Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, New York, NY, USA, 2 edition, 2006. ISBN 978-0-387-30303-1. doi: 10.1007/978-0-387-40065-5.

Roger D Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.

Boris T Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964. doi: 10.1016/0041-5553(64)90137-5.

# END OF TRANSMISSION

**Release:** v3.0.0 · DOI `10.5281/zenodo.20417136` · SHA-256 `pending…` · pairing pending




Figure 9: Integrity QR strip

**Prior:** *No prior releases.*